Apache Spark
Structured Streaming

Kodjo Klouvi | kodjo@osekoo.com

1- Introduction to Structured Streaming

"Stream processing with Spark made simple"

What is Data Streaming?

- Data is generated continuously (Logs, sensors, financial data, etc.)
- Streaming = processing data as it arrives, not after

Streaming vs Batch Processing

Feature	Batch Processing	Streaming Processing
Input Data	Bounded (static)	Unbounded (infinite)
Latency	High	Low
Processing	Periodic	Continuous or micro-batch
Use Case Example	Daily reports	Real-time dashboards

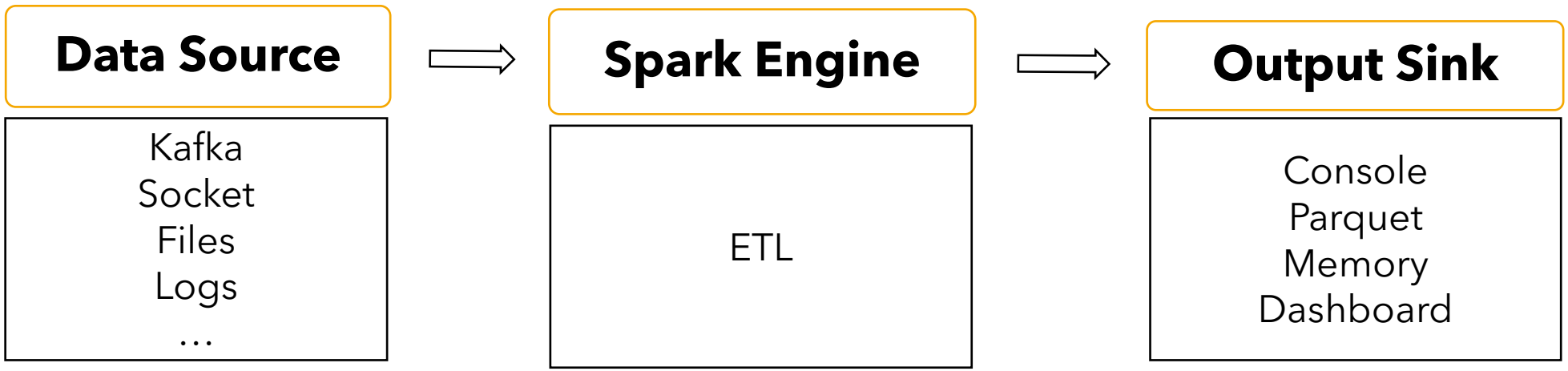
What is Structured Streaming?

- A high-level Streaming API built on Spark Core and Spark SQL
- Treats streaming data a continuous (unbounded) table
- Uses familiar DataFrame/Dataset API
- Support SQL queries, aggregations, joins, windows, etc.

Why Structured Streaming?

- Unified programming model: same code for batch & streaming applications
- Declarative: just write queries (tell Spark what to do), Spark handles execution
- Scalable: leverage Spark's distributed engine
- Fault-tolerant: automatic recovery, state management

Structured Streaming Architecture



Key Terms

- **Input Source**: Streaming data source (Kafka, files, logs, socket, etc.)
- **Output Sink**: Where results are written (console, storage, dashboard, etc.)
- **Trigger**: Define when to process data
- **Watermark**: Helps manage late data
- **State**: Used for aggregations and joins over time (*event-time*)

2- Structured Streaming – Core Concepts

"Stream processing with Spark made simple"

- Common and unified API for batch and stream processing
- Use of SQL queries to process streaming data
- Cover various of stream processing use cases

Input Sources

- Structured Streaming supports multiple sources of real-time data
  - **Kafka**: real-time messaging system (most used)
  - **Socket**: simple line-based text stream (network)
  - **File**: watches folders for new files (file systems, FTP, etc.)
  - **Cloud sources**: Amazon S3, Google Cloud Storage (file)
- Input must be append-only and schema-compatible

DataFrames & Datasets

- Structured Streaming uses **DataFrame/Dataset APIs**:
  - Unified APIs for both **batch and streaming**
  - Streaming queries = **incremental** queries on unbounded Dataframes
  - Can **use Spark SQL, Transformations, UDFs**

Static Schema over Dynamic Data

- Streaming data can evolve, but Spark requires a static schema
- Schema is defined at the start of the streaming query
- This enables:
  - Optimized query planning
  - Serialization/deserialization
- Best practice: define explicit schema instead of inferring

The Unbounded Table Model

- Structured Streaming treats streaming data as a **logical table**:
  - A growing table to which new rows are continuously added
- This allows:
  - Declarative queries (over time)
  - Reuse of SQL concepts (filters, joins, aggregation, etc.)

Micro-Batches
(Default Execution Mode)

- Spark processes data in **small time intervals** (micro-batches)
- Each **micro-batch** reads new data, runs the query, and outputs results
- Example: Trigger every 5 seconds → new batch every 5s
- Scalable – Fault-tolerant – Works well with serval input sources

3- Structured Streaming APIs

“From DataFrames to real-time pipelines”

Reading a Stream: *readStream*

- Use of **spark.readStream** just like **spark.read**
- Specify the input source format
  - kafka
  - Socket
  - csv, json (from directory)

```
val rawKafka = spark.readStream
  .format("kafka")
  .option("subscribe", "clicks")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .load()
```

Writing a Stream: *readStream*

- Use of **spark.writeStream** to start a streaming query
- Call ***.start()*** to launch the streaming job

```
val query = parsed.writeStream
  .format("parquet")
  .option("path", "/output")
  .option("checkpointLocation", "/checkpoint")
  .start()
```

Output Modes

Mode	Description	Use case
Append	Only new rows	Simple ETL, inserts only
Update	Rows that changed	Aggregations without watermark
Complete	All results, every time	Aggregations with full state

Output Sinks

Sink	Usage
console	Debugging/Testing
parquet, json, csv	Persistenct storage
memory	Querable in Spark SQL (testing)
kafka	Push results back to kafka topic
foreach	Custom sink (any side-effect)

4- Execution Model in Structured Streaming

"Understanding how Spark runs streaming queries"

Two Execution Modes

- Spark Structured Streaming offers **two execution models**
- Micro-batch (default): Process data in small time intervals
- Continuous (experimental): Process data row-by-row
- Most use cases rely on **micro-batching**

Micro-Batch Mode

- Spark **reads new data in intervals** (e.g. every 5 seconds)
- Each **micro-batch = mini batch job**
- Fault-tolerance – Scalability – Compatible with most sinks

```
val query = parsed.writeStream
  .trigger(Trigger.ProcessingTime("5 seconds"))
  .format("parquet")
  .option("path", "/output")
  .option("checkpointLocation", "/checkpoint")
  .start()
```

Continuous Mode

- Process records as they arrive, not in batches
- Extremely low latency (~1ms)
- Fewer supported operations
- Only certain sinks (e.g kafka)
- Limited support for stateful operations
- Experimental

```
val query = parsed.writeStream
  .trigger(Trigger.Continuous("1 second"))
  .format("parquet")
  .option("path", "/output")
  .option("checkpointLocation", "/checkpoint")
  .start()
```

Trigger Types

- Triggers define **when the engine processes data**

Trigger Type	Description
ProcessingTime	Every X seconds (e.g. 5s)
Once	One batch, then stop (for testing)
Continuous	Stream continuously with sub-second latency

Watermarking for Late Data

- Late-arriving data is common in event-based systems
- Spark uses watermarks to handle it
- Defines how long to wait for late data
- Works with windowed aggregations and state cleanup

```
val parsed = rawKafka
  .selectExpr("CAST(value AS STRING) as json")
  .select(from_json($"json", schema).as("data"))
  .select("data.*")
  .withWatermark("eventTime", "10 minutes")
```

Stateful Processing

- Structured Streaming can maintain state across batches:
  - Aggregations over time (event-time)
  - Stream-stream joins
  - Deduplication
- But this creates state data:
  - Stored on disk (int the checkpoint directory)
  - Can grow large – must be managed

```
parsed.groupBy(window($"timestamp", "10 minutes"))
  .count()
```

Fault Tolerance and Recovery

- **Checkpointing**: Saves offsets and state metadata
- **Write-ahead logs** (WAL): For durability
- Restarting the query:
  - Restores from last committed batch
  - Resumes exactly-once processing
- Always define ***checkpointLocation*** in ***writeStream***

5- Advanced Topics & Best Practices

"Getting the most out of Structured Streaming"

Performance Optimizations

Key areas to optimize:

- **State Store Management**
  - Reduce window duration
  - Use watermarking to clean up old state
- **Repartitioning**
  - Avoid skew
  - Use .repartition(n) before aggregations
- **Caching**
  - Cache repeated subqueries when using joins or unions

Optimizing Joins

- **Stream-to-Static Join**
  - Works like normal join
  - Static data is broadcasted
- **Stream-to-Stream Join**
  - Requires watermarking
  - Can be expensive
  - Use only when necessary

```
val streamDf = spark.readStream
  .format("kafka")
  .load()

val streamDf2 = spark.readStream
  .format("kafka")
  .load()

val staticDf = spark.read
  .format("parquet")
  .option("path", "/output") Choose s
  .load()
```

```
val streamStaticJoinDf = streamDf.join(staticDf, "id")
  .select(streamDf("value"), staticDf("value"))
```

```
val streamStreamJoinDf = streamDf.withWatermark("timestamp", "10 minutes")
  .join(streamDf2.withWatermark("timestamp", "10 minutes"), "id")
  .select(streamDf("value"), streamDf2("value"))
```

Avoiding Common Pitfalls

Anti-patterns to avoid

- Using **too many shuffle operations** (e.g. ***groupBy*** without partitioning)
- Forgetting to set **checkpointLocation**
- Uncontrolled **state growth**
- **High cardinality** keys in joins or aggregations
- Not monitoring query progress (***query.status***, ***query.lastProgress***)

Monitoring & Debugging

Tools and techniques:

- ***StreamingQuery.status*** and ***StreamingQuery.lastProgress***
- Spark UI: ***/streaming*** tab
- External tools: Prometheus, Datadog, etc.
- Use **structured logs** with ***query.name*** and ***query.explain()***

```
query.explain(true) // print physical plan of the query
```

Testing Streaming Jobs

Testing strategies:

- Use Memory sink for assertions
- Simulate streaming with ***rate*** or ***socket*** source
- Write integration tests using ***awaitTerminationOrTimeout***
- Consider testing **batch-equivalent logic** separately when possible.

```
streamDf.writeStream
  .format("memory")
  .queryName("kafka_stream")
  .outputMode("append")
  .start()
```

Deployment Best Practices

Recommended setup for production

- Set ***spark.sql.shuffle.partitions*** wisely (depends on cluster size)
- Always use ***checkpointLocation***
- Use **rolling log files** to avoid full disk
- Configure Spark for **graceful shutdowns**
- Consider **autoscaling, dynamic allocation**
- *Tip*: Structure your job like a **long-running service**, not a script.

6- Case Study: Reading from Kafka

"Stream ingestion, transformation, and persistence"

Why Kafka?

- Designed for high-throughput, low-latency data ingestion
- Integrates natively with Spark Structured Streaming
- Ideal for:
  - Log collection
  - Real-time analytics
  - Data pipelines
- Kafka provides:
  - Topics = data streams
  - Partitions = scalability
  - Offsets = message tracking

Connect to Kafka

- Kafka data is returned as a DataFrame with columns:
  - key, value → as binary (Array[Byte])
  - topic, partition, offset, timestamp, etc.

```
import spark.implicits._

val rawKafka = spark.readStream
  .format("kafka")
  .option("subscribe", "clicks")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .load()
```

Parse Kafka Messages

- Cast and parse the input message (event)

```
val parsed = rawKafka
  .selectExpr("CAST(value AS STRING) as json")
  .select(from_json($"json", schema).as("data"))
  .select("data.*")
```

Example: Schema for JSON

- Aggregations, filtering, windowing, etc.

```
val schema = new StructType()
  .add("userId", StringType)
  .add("page", StringType)
  .add("ts", TimestampType)
```

```
parsed.groupBy("page")
  .agg(
    count("userId").as("click_count"),
    min("ts").as("first_click"),
    max("ts").as("last_click")
  )
```

Writing to Sink

- Write the transformed data to Parquet

```
val query = parsed.writeStream
  .format("parquet")
  .option("path", "/output")
  .option("checkpointLocation", "/checkpoint")
  .start()
```

Full Example

```
val rawKafka = spark.readStream
  .format("kafka")
  .option("subscribe", "clicks")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .load()

val schema = new StructType()
  .add("userId", StringType)
  .add("page", StringType)
  .add("ts", TimestampType)

val parsed = rawKafka
  .selectExpr("CAST(value AS STRING) as json")
  .select(from_json($"json", schema).as("data"))
  .select("data.*")

val query = parsed.writeStream
  .format("parquet")
  .option("path", "/output")
  .option("checkpointLocation", "/checkpoint")
  .start()
```

7- Hands-on Lab

https://github.com/osekoo/hands-on-spark-streaming

“Think of Structured Streaming as SQL over time.
Focus on writing clean queries.
Then let Spark handle the execution plan.”